Customer Behavioral Analysis & Loan Default Classification

Project Proposal - Implementation Plan Only

PKDD'99 Czech Bank Dataset | Units III & IV: Data Mining, Preprocessing and Classification

Second Jury

This proposal describes HOW the project will be built: what data will be read, what cleaning and feature-engineering code will be written, and what analysis, modeling and segmentation procedures will be implemented.

It does not report any results, metrics, or findings. Those will come only after implementation.

Proposed pipeline: 01 Preprocessing -> 02 Feature Engineering -> 03 Exploratory Analysis -> 04 Classification Modeling -> 05 Customer Segmentation

Stack: Python, pandas, scikit-learn, imbalanced-learn, matplotlib, seaborn, Jupyter

1. Objective of the Proposed Work

We propose to build a loan-default classification system that predicts whether a granted loan will end in default, using only the customer's banking behaviour observed BEFORE the loan was granted. The focus of this proposal is implementation: what steps we will code, in what order, and with what logic. Every step below is described as a procedure to be written, not as a result already obtained.

What we will implement

- A preprocessing module that loads the raw tables, checks data quality, and defines the binary target.
- A feature-engineering module that builds one pre-loan behavioural record per loan, with a strict no-leakage rule.
- An exploratory-analysis module that inspects distributions, correlations and behaviour differences.
- A classification module that trains and compares six standard classifiers under two imbalance settings, with tuning and repeated cross-validation.
- A segmentation module that groups borrowers by behaviour using KMeans, without using the target.

What this proposal does NOT contain

No accuracy, precision, recall, F1, ROC-AUC, correlation values, charts from runs, or selected-model claims. Those are outputs of execution. This document only commits to the procedure we will implement. Where an example threshold or plot type is mentioned, it refers to code we will write, not to a number we have measured.

How to read this document

Sections 2 and 3 fix the problem definition and the input tables. Sections 4 to 8 specify the five notebooks to be written (01 to 05): what each file will read, what functions and pipelines it will contain, what asserts will guard it, and what artefacts it will save. Section 9 lists the software stack and file flow. Section 10 gives the staged work plan, and Section 11 lists the deliverable files to be produced upon implementation.

2. Scope and Implementation Principles

In scope

- Loading five raw CSVs with sep=';' and verifying duplicates, nulls and date parsing.
- Deriving gender and age from birth_number with vectorised string logic and range checks.
- Building 13 pre-loan behavioural features per loan through a single helper function with a strict date filter.
- Joining each loan to its OWNER client for demographics; writing one clean table loan_features.csv.
- Implementing EDA routines (distributions, correlations, boxplots, grouped rates, quadrant tabulation logic).
- Implementing classification comparison (6 models x 2 imbalance arms), tuning, and repeated cross-validation for selection.
- Implementing behaviour-only KMeans segmentation with silhouette-based k selection and profile tables.

Out of scope for this proposal

- No model deployment, no web app, no real-time scoring - only notebook implementation and saved artefacts.
- No external data, no new labelling - only the PKDD tables as given.
- No results discussion - metrics and figures belong to the post-implementation report, not here.

Three principles every notebook will follow

- Leakage-first: any use of transaction history is filtered by trans.date < loan.date before aggregation.
- Asserts-as-proof: each notebook ends with assert statements that fail loudly if joins fan out, counts drift, or metrics leave [0,1].
- One-way flow: 01 -> 02 -> loan_features.csv -> 03/04/05. Later notebooks never re-read raw tables for behaviour.

3. Problem Framing and Target Definition (To Be Implemented)

The source database records loans with a status code in {A, B, C, D}. We will implement the standard PKDD interpretation in code: statuses A and C mean the loan ran to a good end (contract finished, or running without problems), while B and D mean the loan entered default (contract finished with problems, or currently in debt).

```
loan['target'] = loan['status'].map({'A': 0, 'C': 0, 'B': 1, 'D': 1})
# 0 = good, 1 = default (to be implemented in 01_preprocessing)
```

The implementation will also derive two demographic fields from the client's birth_number: gender and age. The birth_number encodes month with a +50 offset for females (a national ID convention): if the middle two digits are greater than 50, the client is female and 50 is subtracted to get the true month. Age will be computed against the reference year of the dataset. Both derivations will be implemented as vectorised pandas string operations, followed by range asserts.

Leakage rule (applies to the whole project)

For every loan, only transactions with trans.date strictly earlier than that loan's grant date may be used. Post-loan transactions will never be read. This single rule will be enforced inside a helper function and verified by asserts, so the classifier can only learn from history that would have existed at decision time.

4. Input Data To Be Used

All inputs are the raw PKDD'99 CSV files stored under data/. Files will be read with pandas read_csv using sep=';'. No external data will be introduced.

Tables and columns we will read

- loan.csv: loan_id, account_id, date (grant date as YYMMDD), amount, duration, payments, status. One row per loan (682 rows). This is the anchor table.
- trans.csv: trans_id, account_id, date, type, operation, amount, balance, k_symbol, bank, account. The behaviour source. Will be filtered to pre-loan rows only.
- account.csv: account_id, district_id, frequency, date. Account metadata for joins.
- disp.csv: disp_id, client_id, account_id, type (OWNER / DISPONENT). Will be filtered to OWNER so each account maps to exactly one client.
- client.csv: client_id, birth_number, district_id. Source for gender and age derivation described above.

The engineered output of the pipeline will be data/loan_features.csv: one row per loan, joining the loan record, the pre-loan behavioural features, and the OWNER client demographics. Downstream notebooks will read only this file, never the raw tables directly.

5. Implementation Step 01: Preprocessing

File to be written: notebooks/01_preprocessing.ipynb

This notebook will implement loading and quality checks. Each of the five tables will be loaded into a dict of DataFrames, then inspected for duplicated rows and null counts. Date columns in trans and loan will be parsed with format '%y%m%d' with errors='coerce', and any unparseable count plus the

min-max range will be printed for inspection.

Checks and asserts to be coded

- Print duplicates per table via df.duplicated().sum() and nulls via df.isnull().sum().
- Verify loan has exactly one row per loan and status is within {A, B, C, D}.
- Verify the target column has no NaN and contains only {0, 1}, and that its positive count equals the count of B plus D rows.
- Verify derived age falls in a sane range (0-100).

The notebook will not build any behavioural features and will not train any model. Its job is to prove the raw inputs are clean and to fix the target definition that everything downstream depends on.

6. Implementation Step 02: Feature Engineering (Pre-Loan Only)

File to be written: notebooks/02_features.ipynb -> outputs data/loan_features.csv

This is the core implementation of the project. For each loan we will collect only the transactions of the same account with parsed_date earlier than the loan grant date. Transactions will be pre-grouped by account_id and sorted by date into a dictionary, so each loan's history is a simple slice. A function behavior_for(account_id, grant_date) will implement this.

Features the function will compute (13 behavioural columns)

- Counts: n_txn, n_deposit, n_withdrawal; activity rate txn_per_month over the observed pre-loan window.
- Money flows: sum_deposit, sum_withdrawal, net_flow (deposits minus withdrawals).
- Balances: avg_balance, min_balance, max_balance, balance_at_loan (last balance before grant), balance_trend (late-window mean minus early-window mean).
- History length: months_history. Withdrawal types identified by operation in {VYDAJ, VYBER}. Empty history returns zeros / NaN-safe defaults.

Demographics will then be joined: birth_number is decoded to gender and age, disp is filtered to type == 'OWNER' and joined from loan to client. The final frame (loan fields + 13 behaviour columns + gender/age/district) will be written to data/loan_features.csv, one row per loan.

Leakage asserts to be coded

- OWNER account_id is unique (no fan-out on join); every loan has a non-null OWNER client.
- Every loan account exists in trans; final row count equals loan row count; no nulls in behaviour columns.
- Spot-check on a random sample of loans: recompute the pre-loan slice independently and compare

n_txn and sums.

7. Implementation Step 03: Exploratory Analysis Procedure

File to be written: notebooks/03_eda.ipynb -> figures to outputs/figures/

This notebook will implement read-only inspection of loan_features.csv. It will not modify features and will not train models. All figures will be saved to outputs/figures/ with a fixed palette for the two classes.

- Target inspection: bar chart of good vs default counts (to confirm imbalance handling is needed downstream).
- Correlation procedure: select numeric columns, drop identifier/date columns (loan_id, account_id, client_id, district_id, date), compute corr()['target'] sorted, and plot the ranking.
- Multicollinearity check: full feature-feature heatmap to see which pairs (e.g. amount vs payments) move together.
- Behaviour comparison: boxplots of balance and flow columns split by target; age histogram in proportional (fill) mode; grouped default-rate tables for gender and status.
- Risk-quadrant procedure: cross balance_at_loan against loan amount at their medians and tabulate default rate per quadrant (method only; no numbers reported here).
- Data-quality section: missing-value counts, duplicated-row count, describe() table for skew/outlier inspection.

8. Implementation Step 04: Classification Modeling Procedure

File to be written: notebooks/04_modeling.ipynb

This notebook will implement the full model-comparison protocol. Identifier and raw-date columns will be dropped before modeling. The data will be split once with stratified 75/25 train_test_split (fixed seed) so the test set preserves the class ratio. Scaling (StandardScaler) and optional resampling (SMOTE) will be placed INSIDE an imblearn Pipeline, so every fold fits them on training data only.

Models and settings to be implemented (6 x 2)

- Classifiers: DecisionTree, NaiveBayes (GaussianNB), RandomForest, SVM (RBF), KNN, GradientBoosting - each constructed via a model_zoo() dict.
- Two imbalance arms per model: baseline (no resampling) and smote (SMOTE inside pipeline). Total 12 fitted runs on the single split.
- Metrics to be computed per run: accuracy, precision, recall, F1, average_precision, ROC-AUC; confusion matrices, ROC and precision-recall curves per arm.

Tuning and selection procedure to be implemented

- RandomizedSearchCV per model x arm (n_iter=30, inner StratifiedKFold k=5 on TRAIN only, scoring F1). Search grids keyed by pipeline step name (e.g. classifier__max_depth). Test set stays untouched.
- Repeated cross-validation: RepeatedStratifiedKFold (5 folds x 10 repeats = 50 estimates) for every model x arm x variant (default vs tuned) - 24 configurations - ranked by mean CV F1.
- Selection rule: pick the tuned configuration with the highest repeated-CV F1 mean; persist it with joblib/pickle to outputs/models/. Save metrics_comparison.csv, tuning_results.csv, cv_comparison.csv.
- Asserts: row counts of the three CSVs (12, 12, 24), all metrics in [0, 1], CV std non-negative, winner key matches the ranked table.

9. Implementation Step 05: Behaviour Segmentation Procedure

File to be written: notebooks/05_segmentation.ipynb

This notebook will implement descriptive segmentation. Clustering will use ONLY the 13 behavioural columns, standardised with StandardScaler. The target will never be passed to KMeans; it will only be averaged afterwards to comment on each segment.

- Fit KMeans for k = 2..6 (n_init=10, fixed seed) on the scaled behaviour matrix; record silhouette_score and inertia per k.
- Selection rule to be coded: best_k = argmax silhouette. Refit KMeans with best_k and assign f['segment'].
- Profiling to be coded: groupby segment median of behaviour columns plus size and mean target (default rate) per segment; save to segment_profiles.csv.
- Plots to be implemented: default-rate bar per segment and a 2-D scatter (e.g. avg_balance vs txn_per_month) coloured by segment.
- Asserts: no null segments, nunique == best_k, per-segment rates in [0, 1], silhouette of best_k above a weak-structure floor (0.2), best_k equals the argmax.

10. Technology, File Flow, and Execution Order

Libraries to be used

- pandas + numpy: loading (sep=';'), date parsing, groupby slicing, joins, asserts.
- scikit-learn: StandardScaler, Pipeline, train_test_split, StratifiedKFold, RepeatedStratifiedKFold, RandomizedSearchCV, six classifiers, KMeans, silhouette_score, metrics and curves.
- imbalanced-learn: SMOTE inside Pipeline (training-fold only).
- matplotlib + seaborn: bar charts, heatmaps, boxplots, histograms, ROC / PR curves, CV ranking chart.
- jupyter + nbconvert: notebooks executed top-to-bottom in order 01 to 05.

```
pip install -r requirements.txt
# pandas==2.2.3 numpy==2.2.4 scikit-learn==1.7.0
# matplotlib==3.10.3 seaborn==0.13.2 imbalanced-learn==0.14.2
# jupyter==1.1.1 nbconvert==7.17.1
```

File flow to be implemented

```
data/*.csv  ->  01_preprocessing (checks + target)
            ->  02_features -> data/loan_features.csv (1 row / loan)
                        ->  03_eda -> outputs/figures/*.png
                        ->  04_modeling -> metrics_*.csv + models/best_*.pkl + figs
                        ->  05_segmentation -> segment_profiles.csv + figs
```

Execution order is strictly 01, then 02, then 03/04/05 (the last three read only loan_features.csv and are independent of each other). Each notebook will run top-to-bottom without manual intervention, e.g. via nbconvert --execute. This proposal commits only to building the above procedure; evaluation and conclusions belong to the post-implementation report.

11. Staged Work Plan (Implementation Order)

Work will proceed notebook by notebook so each stage can be verified before the next begins. No stage reports numeric findings; each stage ends with files and passing asserts.

- Stage A - Preprocessing (01): write loaders, duplicate/null/date checks, birth_number decoding, target mapping, and asserts. Artefact: cleaned in-memory frames ready for 02.
- Stage B - Feature engineering (02): write prior_by_acct dict, behavior_for() with trans.date < loan.date filter, OWNER join, and leakage spot-checks. Artefact: data/loan_features.csv (one row per loan).
- Stage C - Exploratory routines (03): write distribution, correlation, boxplot, histogram and quadrant-tabulation code saving to outputs/figures/. No modelling here.
- Stage D - Modeling pipeline (04): write model_zoo(), Pipeline with scaler and optional SMOTE, single-split comparison, RandomizedSearchCV, repeated CV, selection and persistence. Artefacts: three CSVs, best model pkl, curves.
- Stage E - Segmentation (05): write scaling, KMeans k=2..6 sweep, silhouette selection, profiling and plots. Artefact: segment_profiles.csv plus figures.

Each stage will be executed top-to-bottom via Jupyter or nbconvert --execute. A stage is considered done only when its asserts pass and its expected files are written, not when any particular metric value appears.

12. Deliverables To Be Produced (Files, Not Findings)

Upon approval, implementation will produce the following files. They are listed here as build targets, without any claimed contents or values.

- notebooks/01_preprocessing.ipynb, 02_features.ipynb, 03_eda.ipynb, 04_modeling.ipynb, 05_segmentation.ipynb - the five implementation modules.
- data/loan_features.csv - one row per loan: loan fields plus 13 pre-loan behaviour columns plus OWNER demographics.
- outputs/figures/*.png - EDA and modeling plots (distributions, correlations, boxplots, curves, segment charts).
- outputs/metrics_comparison.csv, tuning_results.csv, cv_comparison.csv - modeling comparison tables for later selection.
- outputs/models/best_*.pkl - persisted winning pipeline (scaler plus classifier) for reuse.
- outputs/segment_profiles.csv - median behaviour and size per KMeans segment.

A separate final report (not this proposal) will present the measured outputs after execution. This proposal ends at the point where the above files exist and all asserts pass.

End of proposal. Next step after approval: implement notebooks 01-05 exactly as specified above, then report outputs separately.